interface [Very important for Spring framework] :

Interface upto JDK 1.7V

* An interface is a keyword in java which is similar to a class because .class file will be generated by java compiler.

* An interface defines the **"Working functionality of a class"**.

```
interface iif
{
    void display();
    void display();
    void display();
    void display();
    void display();
}
```

* Upto JDK 1.7V, Inside an interface, we can accept only abstract methods that means we cannot write any concrete OR typical execution.

* From JDK 1.8 we can also write default & static methods inside an interface.

* Inside an interface all the methods are by default **public & abstract.**

* All the variables are by default **public, static and final [Initialization is compulsory at the time of dec.]**

* Interface methods are by default **public** or at the time of overriding, we aren't reduce the visibility.

* We are achieve 100% abstraction.

* An interface cannot have instance variable,constructor and any kind of initializer(static OR non static block)

* An interface defines "WHAT TO DO" and its implementer classes defines "HOW TO DO"

* **It does not support constructor** as we can achive multiple inheritance in java.
  _Don't ask that interface code is used to support multiple inheritance.

* The main purpose of interface is provide **"Loose Coupling"**

* In order to implement the member of interface we should use "implements" keyword.

* We cannot create an object for interface [Instatiate is a an abstract class]

* Overriding all the abstract methods of an interface is compulsory otherwise sub class will become abstract class

//Program :

```
package com.rvs.interface_ex1

abstract interface iif
{
    void display();
    void display();
}

public class InterfaceDemo1
{
    public static void main(String[] args)
    {
        iif iif=iif1 = new iif1();
        iif1 ii1=new iif1();
            @Override
            public void display()
            {
                IO.println("Key one is pressed");

            }

            @Override
            public void display()
            {
                IO.println("Key two is pressed");

            }
        };

        iif1.display();  ii1.display();

    }

}
```

```
package com.rvs.interface_ex2

public interface Movable implements iif
{
    int SPEED = 120; //int() = static + final
    void move();     //int() = abstract

final class Car implements Movable
{
    @Override
    void move()
    {
        //int() = 120; //final
        IO.println("Car is moving with "+SPEED+" KM/HR Speed");
    }

}

public class InterfaceDemo2
{
    public static void main(String[] args)
    {
        Movable movable = new Car();
        movable.move();
        IO.println("Speed of the Movable object is - "+Movable.SPEED+" KM/HR Speed");

    }

}
```

```
package com.rvs.interface_ex3

interface Bank
{
    double getBalance();
    void deposit(double amount);
    void withdraw(double amount);

class Customer implements Bank
{
    protected double Balance = 10000;
    private String name;

    public Customer(String name)
    {
        super();
        this.name = name;
    }

    @Override
    public double getBalance()
    {
        return this.Balance;
    }

    @Override
    public void deposit(double amount)
    {
        if(amount <=0)
        {
            System.err.println("Deposit amount cannot be zero OR negative");
            System.exit(0);
        }
        this.Balance = this.Balance + amount;
    IO.println("After deposit "+amount+" amount, Balance is :"+this.getBalance());
    }

    @Override
    public void withdraw(double amount)
    {
        if(amount > this.Balance)
        {
            System.err.println("Insufficient Balance");
            System.exit(0);
        }
        this.Balance = this.Balance - amount;
    IO.println("After withdraw "+amount+" amount, Balance is :"+this.getBalance());
    }

}

public class InterfaceDemo3 {

    public static void main(String[] args)
    {
        Bank cust = new Customer("Raj");
    IO.println("Initial Balance is :"+cust.getBalance());
        cust.deposit(5000);
        cust.withdraw(8000);
    }

}
```

//Program on Loose Coupling :

```
//Program on Loose Coupling :
```